



Tecnológico
de Monterrey

Reflexión

Alumno: Diego Javier Martínez Sánchez

Matrícula: A00845422

Materia: Estructura de datos y algoritmos fundamentales

Grupo: 607

Profesor: David Alonso Cantú

Fecha de entrega: 22 de septiembre de 2026

Importancia de ordenar y buscar registros

Antes de realizar esta actividad, yo veía el ordenamiento principalmente como un tema de algoritmos y complejidades. Al trabajar con los logs entendí una aplicación más concreta: un registro de seguridad aislado dice poco, pero al poner miles de eventos en orden cronológico se puede reconstruir qué ocurrió antes, durante y después de un incidente. Esto sirve, por ejemplo, para detectar intentos repetidos de acceso, identificar una secuencia de errores o revisar la actividad de una dirección IP en un periodo específico.

La búsqueda por rango también fue importante porque evita revisar manualmente todo el archivo. Una persona que investiga un evento normalmente conoce una fecha aproximada, no el número exacto de línea donde aparece. Con los registros ya ordenados, la búsqueda binaria permite localizar los extremos del periodo en $O(\log n)$ y después mostrar solamente los elementos relevantes. Para este proyecto definí que el rango fuera inclusivo: si el límite inicial o final coincide con registros duplicados, todos deben mantenerse. Esa decisión fue necesaria porque los duplicados no son errores del archivo; pueden representar eventos distintos con el mismo timestamp.

Comparación de los algoritmos

Implementé Swap Sort, Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort y Shell Sort. La diferencia más clara entre ellos fue que no todos aprovechan de la misma forma que un archivo esté casi ordenado. Selection Sort fue un buen ejemplo: aunque el archivo tuviera pocas inversiones, el algoritmo siguió buscando el mínimo restante en cada posición, por lo que sus tiempos fueron parecidos. En mis mediciones pasó de 131,172 microsegundos en el archivo desordenado a 113,377 microsegundos en el casi ordenado.

Insertion Sort mostró el comportamiento contrario. En el archivo desordenado tardó 6,881,336 microsegundos, mientras que en el casi ordenado tardó 10,270 microsegundos. La diferencia se explica porque en una lista casi ordenada cada elemento suele estar cerca de su posición final y necesita pocos desplazamientos. Por eso su mejor caso es $O(n)$, aunque su peor caso siga siendo $O(n^2)$.

Merge Sort fue muy consistente: 68,011 y 66,309 microsegundos. Esto coincide con que mantiene $O(n \log n)$ en mejor y peor caso. Quick Sort fue el más rápido en mis dos archivos, con 17,658 microsegundos en el desordenado y 7,132 en el casi ordenado. No significa que siempre será la mejor opción, porque su peor caso es $O(n^2)$, pero el pivote central funcionó bien con estos datos. Shell Sort también tuvo buenos resultados, aunque con una mejora más moderada.

Interpretación de las mediciones

Las predicciones que hice antes de cada corrida en general coincidieron con los resultados. Esperaba que Insertion Sort y Bubble Sort se beneficiaran del archivo casi ordenado, y ambos mejoraron mucho. También esperaba que Merge Sort se mantuviera estable, lo cual ocurrió. Un resultado que me pareció interesante fue Swap Sort: conserva una cantidad cuadrática de comparaciones, pero pasó de 7,261,659 a 173,415 microsegundos. La complejidad no cambió, pero al haber menos intercambios también disminuyó el costo real de mover objetos Log que contienen strings.

Esta actividad me ayudó a distinguir entre complejidad teórica y desempeño observado. La complejidad permite anticipar cómo crece el trabajo al aumentar n , pero el tiempo final también depende del orden inicial, de cuántos intercambios se realizan y de los costos propios de los datos que se mueven. Por eso no basta con afirmar que un algoritmo es mejor; se debe relacionar la teoría con el contexto y con mediciones reales.

Uso crítico de inteligencia artificial

Usé inteligencia artificial como apoyo durante el desarrollo y para la elaboración de los documentos de evidencia. La utilicé para revisar funciones específicas, entender errores de compilación y enlazado, organizar el documento de pruebas y contrastar decisiones de implementación. Sin embargo, cada cambio se incorporó de manera incremental, se compiló y se probó antes de guardarlo en el repositorio.

Tres consultas importantes fueron: "¿Cómo puedo construir una llave numérica para comparar fecha y hora en un registro de log?"; "¿Por qué aparece el error undefined reference al compilar main.cpp junto con otros archivos?"; y "¿Cómo puede Bubble Sort aprovechar un archivo casi ordenado?". A partir de estas consultas adapté la estructura Log para crear una llave cronológica, corregí el comando de compilación para incluir todos los archivos .cpp y revisé la condición de parada de Bubble Sort.

El caso más claro de revisión crítica fue precisamente Bubble Sort. La primera versión que utilicé completaba todas las pasadas incluso si la lista ya estaba ordenada. No era un error que produjera un orden incorrecto, pero sí era una sugerencia ineficiente para el objetivo de comparar un archivo desordenado contra uno casi ordenado. Lo detecté al revisar la condición de mejor caso: sin una variable que registre si hubo intercambios, el algoritmo seguía siendo $O(n^2)$ incluso cuando no había nada que mover. La corrección fue agregar una bandera swapped y terminar cuando una pasada no realiza intercambios. Después de eso Bubble Sort pasó de 7,693,272 microsegundos en el archivo desordenado a 13,122 microsegundos en el casi ordenado.

También utilicé IA para dar formato al documento EvidenciasPruebas, pero los resultados, las capturas y la interpretación de las mediciones corresponden a las pruebas que ejecuté

en mi programa. Considero que la IA fue útil cuando sirvió para cuestionar y verificar una decisión, no cuando se toma una respuesta como correcta sin entenderla.

Conclusión personal

Al terminar la actividad, considero que el aprendizaje principal fue entender que ordenar no es un paso aislado: prepara los datos para buscar, analizar y tomar decisiones con mayor rapidez. También confirmé que el estado inicial de los datos cambia mucho la elección del algoritmo. Para un log grande y desordenado elegiría Quick Sort o Merge Sort según la necesidad de consistencia, mientras que para un archivo casi ordenado consideraría Insertion Sort como una opción muy eficiente. Finalmente, el uso de pruebas, commits y documentación me ayudó a comprobar no solo que el programa funciona, sino por qué funciona y en qué casos conviene cada alternativa.